Large-Scale Analytics with Spark

Profiling and Optimising a PySpark Pipeline over the NYC Yellow Taxi Records

Miguel Lança
Pedro Marteleira
TODO@domain.com
TODO@domain.com

ABSTRACT

This report describes the implementation, profiling and optimisation of a PySpark pipeline over the NYC TLC Yellow Taxi trip records, this pipeline ingests and cleans the raw records, calculates the temporal demand per borough and monthly statistics per zone with window functions, and implements three of the optional modules: fare anomalies, demand forecasting features and trip type classification, all experiments performed were conducted on a Spark 4.2 standalone cluster running with Docker on a single laptop, over the complete year of 2019 (with 84.6 M records). Starting from a very simple baseline, we identified three bottlenecks in the query plans and event logs, namely the recomputation of the input, the shuffle of the whole trip table and a join placed before an aggregation, and then we addressed each one with a measured change, the final version of the pipeline is 42% faster than the baseline and shuffles 97% less data. The strong scaling, on the other hand, is limited by the environment itself, since every worker shares the same four physical cores of the machine.

KEYWORDS

Apache Spark, PySpark, performance profiling, query optimisation, scalability

1 INTRODUCTION

Apache Spark [**?**] is nowadays the reference engine for the processing with large datasets, in great part due to the powerful DataFrame API of Spark SQL [**?**], whose optimiser decides the most of the physical execution, however a correct Spark program is not necessarily an efficient one, since the same analysis may read the input several times, move gigabytes across the network or run hundreds of tiny tasks, depending on decisions that are not visible in the source code. In this work assingment we implemented a pipeline over the NYC TLC Yellow Taxi trip records [**?**], composed of the mandatory operations (M1 to M3) and three of the optional operations (O1, O3 and O4). We followed a strict methodology: first, a simple baseline thought AI agents, without caching, broadcast hints or configuration tuning, then the profiling of the baseline with the Spark UI, the physical plans and the event logs, and only after that the optimisations, one at a time, and each one measured against the previous version.

Section **??** describes the implementation, Sections **??** and **??** the profiling and the optimisations, Section **??** the scalability analysis, and Section **??** concludes.

2 PIPELINE DESIGN

This pipeline is organised as one Python package per operation, and a single entry point only creates the session, calls each operation and writes its result, there are two rules that were followed throughout the code: firstly, the modules contain no actions (`count`, `collect`, `show`): each operation ends in exactly one Parquet write and, since everything before it is lazy, the duration of that write is the duration of the operation, secondly, no Python User-defined-functions (UDFs) are used, so every expression can be optimised by Catalyst and no data has to be serialised between the JVM and the Python workers. The session time zone is set to UTC, to match the timestamps stored in local New York time, and are never shifted by the time zone of the executors.

Ingestion and cleaning (M1). The schema of the files is not stable across the years, since some columns change the case of their name (i.e. `airport_fee`) or a change from an integer to double, and a single `spark.read.parquet` over all the files fails, for this reason the files are grouped by the physical schema (only the Parquet footers are read), each group is mapped onto a canonical schema of 20 columns by a case-insensitive name match followed by an explicit cast and the groups are combined with `unionByName`, the surcharges that did not exist in the earlier years are set to zero, since their absence means that they are not charged, and the files of 2009 to 2010, which have coordinates instead of location IDs, are discarded. A record is invalid when it violates at least one of five rules, null values counting as violations: non-positive fare or distance, `passenger_count` outside 1–6, drop-off not after the pickup, and pickup or drop-off location outside 1–263 (264 and 265 are the *unknown* and *outside of NYC* zones). The rules of the validations are kept as SQL conditions in a dictionary, so the filter and the per-rule statistics (one pass, behind a `--stats` flag) share the same definition. In the year of 2019, about 80.8 M of the 84.6 M records (95.5%) are valid.

Temporal demand and zone statistics (M2 and M3). The M2 joins the trips with the zone lookup on the pickup side and computes, per (`pickup_borough`, `day_of_week`, `hour_of_day`), the number of trips and the mean fare, duration and distance and the output is partitioned by borough. In the M3, a *month* is a calendar month of a given year, so for example January 2019 and January 2020 are different. For the per (zone, month) we compute the mean and sample standard deviation of the fare and distance and the rank of the zone by volume inside its borough using window functions, the rolling 3-month trip share, i.e. the trips of the zone in the current and two previous months over those of its borough in the same period, and an anomaly flag raised when the volume deviates from the mean of the three *previous* months by more than 1.5 standard

deviations. The windows use `rangeBetween` over a month index instead of `rowsBetween`, so a *month* without trips is not silently skipped.

Optional operations (O1, O3 and O4). The O1 calculates the fare per mile of every trip and its mean and the standard deviation per (pickup zone, hour), a trip is anomalous when it exceeds the mean by more than 3σ, these statistics are joined back to every trip, and the anomaly rate is aggregated by the day type (weekday or weekend), payment type and borough. The O4 computes per (pickup zone, hour), the fraction of airport trips (`RatecodeID` 2 or 3), of the negotiated fares (`RatecodeID` 5) and of group trips (4 or more passengers), and labels the bucket as *transit*, *commercial* or *residential* when the respective fraction reaches a configurable threshold, or as *mixed* otherwise. The thresholds (0.10, 0.35 and 0.12) are close to the 90th percentile of each fraction in January 2019, so the top 10% of the buckets receive each label, the label is joined back to every trip, and the trips are aggregated per zone type and month. Finally, the O3 counts the trips per (zone, date, hour) and calculates the mean and variance of these counts per (zone, day of week, hour) over the loaded period, the count of four weeks before is obtained through a self-join with the date shifted by 28 days (a left join, since the first four weeks have no lag), the slots whose count is more than twice the bucket mean are flagged as demand spikes, and each slot is annotated with the `service_zone` of its zone, the hours without any trips haven't any row, which is a simplification since the bucket means of the quiet zones are slightly overestimated.

3 PROFILING AND BASELINE

Experimental setup. Since the DI cluster wasn't available, every experiment was performed in a single laptop (with an Intel i5-8365U, 4 cores / 8 threads, 16 GB of RAM, Windows 11) with a Spark 4.2.0 standalone cluster in Docker (WSL2, Java 21), composed with one master and two workers with 2 cores each, i.e. 4 task slots. As allowed by the assignment in this situation, the input is the complete year of 2019 (12 files, 1.2 GB of Parquet, 84.6 M records), each executor has 2 GB of memory (Section **??**) and the only non-default setting is `spark.sql.optimizer.canChangeCachedPlanOutputPartitioning=true` (Section **??**). The stage metrics (i.e.tasks, shuffle bytes, spill and task-time skew) were extracted from the event log of every run, the versions were measured in interleaved rounds (A, B, C, A, B, C, …), with the computer to the mains, even so, the run-to-run spread reaches ±10%, we speculate that this is due to the thermal behaviour of this laptop, and smaller differences are not claimed as gains, all times are medians of three runs.

Baseline. The baseline executes in **341.7 s** (runs between 314.2 and 370.7 s); Table **??** presents it per operation

The event logs and the physical plans point to these three bottlenecks:

(1) **Recomputation** Since nothing is cached, the scan and the cleaning run 8 times (once in M2 and M3, twice in O1, O4 and O3), and 676.8 M records are read for an input of 84.6 M.

Table 1: Baseline per operation (2019, with 2 workers)

Op	Time (s)	Tasks	Records read (M)	Shuffle write (MB)
M2	45.1	14	84.6	0.7
M3	40.3	18	84.6	1.4
O1	85.1	53	169.2	1,320.9
O4	83.1	38	169.2	1,284.8
O3	76.4	48	169.2	98.9
Total	341.7	184	676.8	2,707

(2) **Shuffle of the whole trip table** O1 and O4 join every trip back to a table of ~6 k (zone, hour) statistics, the initial plan is a `SortMergeJoin`, the AQE converts it into a `BroadcastHashJoin` at runtime, but only after the trip side (80.8 M rows) has already been written to the shuffle, i.e. 1.3 GB per operation and 96% of all the shuffled bytes (Figure **??**).
(3) **Join before aggregation on M3** The zone lookup is joined to all the 84.6 M trips before the `groupBy`, although the borough and the zone only depend on `PULocationID`.

The other usual symptoms were also verified and not found: the scan reads only the 9 columns that are used (out of 20), with the `IsNotNull` filters pushed down, no stage spills to disk, and there is no skew, since in every stage longer than 0.5 s the slowest task takes at most 2.0× the median, should also be noted that every 2019 file is a single Parquet row group of 6–8 M rows, so the scan always has 12 tasks for 4 slots (3 waves), independently of the value of `maxPartitionBytes`.

4 OPTIMISATION STUDY

Each optimisation below follows the same structure, since it involves the observed bottleneck, the hypothesis, the change and the measured result, the Table **??** summarises the cumulative effect.

4.1 Executor memory

With the default executors of 1 GB the baseline runs, but every version that caches the trips fails with an `OutOfMemoryError` while the cache is being built (`MemoryStore.putIterator`), after 4 retries of the same task, our suspicion was that each task caches a whole month (one row group, ~7 M rows) and that two tasks share the same heap. With `–executor-memory 2g` for all the versions, the baseline included, the cached versions complete, this setting is used in every runs in this report.

4.2 Caching the cleaned trips

Persisting the cleaned trips (`MEMORY_AND_DISK`) should replace 7 of the 8 scans by the reads in the cache, the first attempt was persisting the DataFrame as is, *failed*, since the cached rows still hold all the 20 columns and the executors run out of heap even with 2 GB, selecting the 8 columns used by the analyses before persisting solves the problem since the cache takes 1,463 MB, entirely in memory and the input is readed only once, the wall time dropped from 341.7 to **207.9 s (-39%)**. The M2 becomes slower (45.1→92.4 s), since it now pays for building the cache, where the following operations become 1.8–3.5× faster (M3 40.3→22.6, O1 85.1→33.1, O4

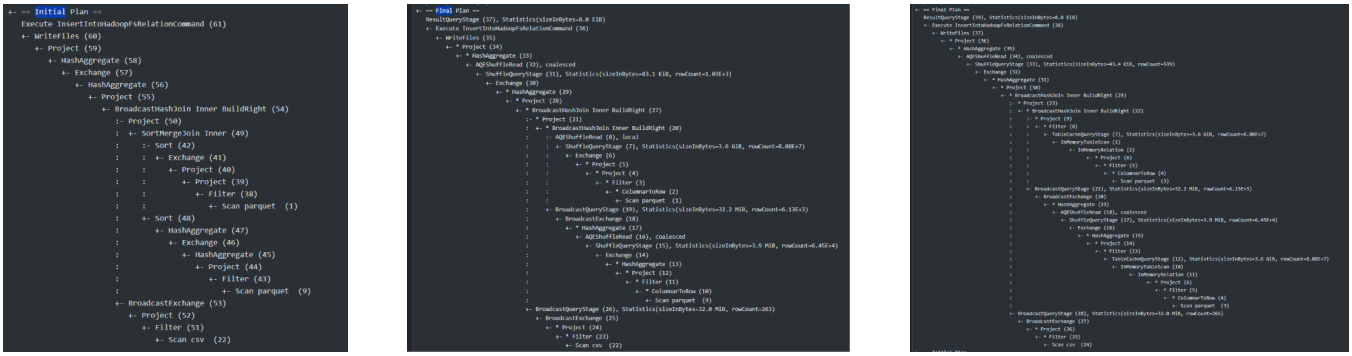


Figure 1: O1 plans in the Spark UI, Left: initial baseline plan, a `SortMergeJoin` with an `Exchange` on both sides and two `Scan parquet`. Centre: baseline plan after AQE, the join became a `BroadcastHashJoin`, but the trip side had already been shuffled (`ShuffleQueryStage (7)`: 80.8 M rows, 1,317.8 MiB in 200 partitions). Right: the final plan, the trips come from the cache (`InMemoryTableScan`) straight into the `BroadcastHashJoin`, and only the aggregated rows are shuffled (3.9 MiB and 43 KiB).

83.1→26.7, O3 76.4→21.8 s). As the first attempt never completes, this measurement also includes the broadcast joins of Section **??**, which were introduced together with it.

4.3 Broadcast joins in O1, O4 and O3

As seen in Section **??**, AQE already converts the join of O1 and O4 into a broadcast, but too later, since the statistics are an aggregate, so their size is only known after the stage that computes them, and the trip side is shuffled in the same wave, since this table has at most 265 zones × 24 hours, we broadcast it from the start, in O1, in O4 and on the bucket statistics of O3. The shuffle writes drop to 3.1 MB in O1 and 1.5 MB in O4, and in the whole pipeline from 2,707 to 72 MB (-97%), the time gain is part of the -39% above, the right side of Figure **??** shows that the trips no longer cross any `Exchange`.

4.4 Aggregating before the join in M3

Grouping first by `(PULocationID, month)` and only then joining the 265-row lookup to the ~3 k aggregated rows gives the same result with a much smaller join, with this change, the M3 went from 22.6 to **13.6 s (-40%)**. The wall time goes from 207.9 to 196.7 s, which is within the run-to-run spread, so only the gain in the operation itself is claimed.

4.5 Re-planning cached plans with AQE

The O3 caches its hourly counts, the output of a shuffle, and reads them three times (bucket statistics, current slots and 4-week lag), the pipeline enables `canChangeCachedPlanOutputPartitioning` (`false` by default), and we expected it to have no effect, since the main cache comes directly from the scan, the hypothesis was wrong, without this, the cached counts keep the 200 default shuffle partitions, because AQE may not coalesce a cached plan, and O3 runs 6 stages of 200 tasks of 20–400 ms each (1,217 tasks instead of 44, for 5.6 MiB of input, Figure **??**). With this, the O3 takes 67.1 s instead of 22.5 s, and the pipeline 236.8 s instead of 199.4 s, so the setting was kept. Lastly, the pipeline also enabled AQE, partition coalescing, skew join and the local shuffle reader explicitly, since Spark 4.2 already uses these values by default and the profile shows no skew, they were removed, without any change in the wall time (196.7 vs. 199.4 s).

Table 2: Optimisations, cumulative (2019, 2 workers, 2 GB executors)

Version	Wall (s)	vs. baseline	Shuffle (MB)
Baseline	341.7	–	2,707
Cache all columns	OOM	–	–
Cache 8 columns + broadcast	207.9	-39.2%	73
+ M3 aggregate first	196.7	-42.4%	72
+ AQE defaults removed (final)	199.4	-41.6%	72
Final without cached-plan AQE	236.8	-30.7%	67

In a nutshell, the final version reads the input once, shuffles 97% less bytes and runs in 199.4 s, witch is **42% faster** than the baseline, the largest remaining cost is the stage that scans, cleans and caches the input (about one third of the wall time), which is bounded by 12 non-splittable row groups on 4 slots.

5 SCALABILITY ANALYSIS

Both experiments use the final version, and the Docker VM was given 12 GB instead of 7.7 GB in order to fit 3 workers, with 2 workers and the full year, the final version takes 197.2 s, consistent with the 199.4 s of Table **??**.

Data volume. With 2 workers we get 0.1, 0.2, 0.5 and 1.0 of 2019 take 76.4, 86.2, 123.6 and 197.2 s (Figure **??**, left), showing that ten times more data costs only 2.6 times more time, because there is a large fixed part ($T \approx 63 + 134f$ s). One part of it is the scan itself, since the sampling is applied after the read and every run decodes all the 84.6 M records (the scan stage takes at least 19.5 s even at $f = 0.1$), the other is the cost of running 41 stages, most of them are small, the M3 and O3, which work on aggregated rows are the ones that grow the least (M3 7.4→14.2 s, O3 12.4→22.4 s).

Number of workers. With the full year, a single worker fails in all 3 runs with an `OutOfMemoryError`, as one executor of 2 GB has to

Stage Id ▾	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
54	$anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768	+details	2026/11/05 22:34:36	21 s	200/200	5.6 MiB	26.5 MiB	22.1 MiB	
51	$anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768	+details	2026/11/05 22:34:32	0.6 s	4/4			23.9 MiB	
49	$anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768	+details	2026/11/05 22:34:27	4 s	200/200	5.6 MiB			22.1 MiB
48	$anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768	+details	2026/11/05 22:34:26	5 s	200/200	5.6 MiB			23.9 MiB
47	parquet at DirectMethodHandleAccessor.java:103	+details	2026/11/05 22:34:21	1 s	200/200	5.6 MiB			
46	parquet at DirectMethodHandleAccessor.java:103	+details	2026/11/05 22:34:21	1 s	200/200	5.6 MiB			
45	parquet at DirectMethodHandleAccessor.java:103	+details	2026/11/05 22:34:21	3 s	200/200			15.1 MiB	

Figure 2: O3 stages without `canChangeCachedPlanOutputPartitioning`: 200 tasks per stage for 5.6 MiB of input (~28 KB per task).

hold the whole cache of 1.46 GB while still decoding parquet page, to compare 1, 2 and 3 workers in the same conditions we therefore use $f = 0.5$ (Table **??**), with the full year, 3 workers take 198.9 s against 197.2 s with 2, which is the same picture.

Table 3: Strong scaling (2019, $f = 0.5$, using 2 cores and 2 GB per worker), Scan is the median task of the stage that reads, cleans and caches the inputs

Workers	Slots	Wall (s)	Speedup	Efficiency	Scan task (s)
1	2	139.7	1.00	100%	7.0
2	4	123.6	1.13	57%	11.4
3	6	123.9	1.13	38%	18.0

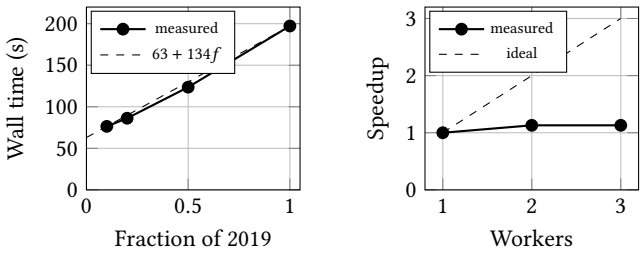


Figure 3: Left: wall time vs. input fraction (2 workers). Right: speedup vs. number of workers ($f = 0.5$).

The speedup is far from the ideal for these three reasons: To start there is no **additional hardware**, all the workers are containers on the same 4-core laptop and the same SSD, so a new worker adds task slots but not cores and this is visible in the scan stage, as the concurrent tasks go from 2 to 4 and 6, the median scan task goes from 7.0 to 11.4 and 18.0 s, and the stage only goes from 46.0 to 38.4 and 37.3 s, since with 6 slots the additional tasks run on hyperthreads of cores that are already busy. Secondly is that, the **partitioning is fixed**, since the scan has 12 tasks (6, 3 and 2 waves), and most of the other 40 stages have 1–12 small tasks, which do not benefit from more slots. The last one is that there is a **fixed overhead**, since the driver plans and schedules 41 stages, one operation after another, and this part, like the 63 s above, does not shrink with more workers. As the run-to-run spread at $f = 0.5$ is about ±5%, the performance of 2 and 3 workers are equivalent, on a real cluster, where each worker brings its own cores, disk and memory, we would expect the scan to scale up to 12 slots, more than that, additional input splits would be needed (files with several row groups, or a `repartition` after the scan).

6 CONCLUSION

The pipeline produces all the required outputs over a year of data in 199 s, against 342 s for the baseline, the largest gains did not come from configuration tuning, but from reading the physical plans and applying the optimizations such as: caching only the columns that are used, broadcasting the small side of a join before the trip side is shuffled, and aggregating before joining, this should also be noted that two of our assumptions were contradicted by the measurements, namely that a cache built from a parquet scan would not need AQE to re-plan its partitioning, and that more workers would reduce the execution time, both showed that in Spark, a change is only an optimisation once it is measured. As future work, the same experiments should be repeated on a real cluster and over the full dataset period, where the scan stage would be no longer be limited by the shared CPU cores.

REFERENCES

[] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, Melbourne, Victoria, Australia, 1383–1394.

[] NYC Taxi and Limousine Commission. 2026. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Accessed November 2026.

[] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A Unified Engine for Big Data Processing. *Commun. ACM* 59, 11 (2016), 56–65.